

Digital Design and Computer Organisation Laboratory

UE23CS251A

3rd Semester, Academic Year 2024-25

Date:03/10/2024

Name: Dipak Rajendra Chaudhari	SRN: PES1UG24CS804	Section K
-----------------------------------	-----------------------	--------------

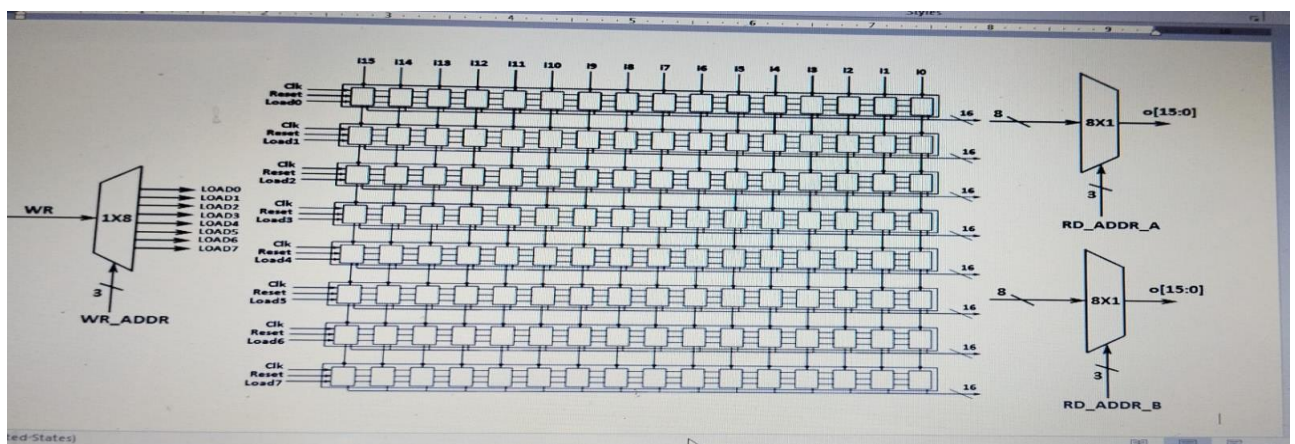
Week# 5

Program Number: 1

TITLE: Register File

Aim: Design a register file with two 16-bit read ports and one 16-bit write port, allowing two values to be read and one value to be written every clock cycle. The register array stores 16-bit data, typically with multiple registers. The read and write operations occur simultaneously.

Circuit Diagram:-



Codes:-

```
`timescale 1ns / 1ps

module tb_register_file();

    // Testbench Signals
    reg clk;
    reg rst;
    reg write_en;
    reg [2:0] wr_addr;
    reg [2:0] rd_addr;
    reg [15:0] wr_data;
    wire [15:0] rd_data;

    // Instantiate the Register File
    register_file UUT (
        .clk(clk),
        .rst(rst),
        .write_en(write_en),
        .wr_addr(wr_addr),
        .rd_addr(rd_addr),
        .wr_data(wr_data),
        .rd_data(rd_data)
    );

    // Clock Generation: 10ns period
    initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end

    // Initialize and Stimulate Inputs
    initial begin
        // Initialize Dump for GTKWave
        $dumpfile("tb_register_file.log");
        $dumpvars(0, tb_register_file);
    end
endmodule
```

```
module register_file (
    input wire clk,
    input wire rst,
    input wire write_en,
    input wire [2:0] wr_addr,
    input wire [2:0] rd_addr,
    input wire [15:0] wr_data,
    output reg [15:0] rd_data
);

    reg [15:0] registers [7:0];

    // Read process
    always @(*) begin
        if (rst) begin
            rd_data = 16'b0;
        end else begin
            rd_data = registers[rd_addr];
        end
    end

    always @(posedge clk) begin
        if (rst) begin
            registers[0] <= 16'b0;
            registers[1] <= 16'b0;
            registers[2] <= 16'b0;
            registers[3] <= 16'b0;
            registers[4] <= 16'b0;
            registers[5] <= 16'b0;
            registers[6] <= 16'b0;
            registers[7] <= 16'b0;
        end else if (write_en) begin
            registers[wr_addr] <= wr_data;
        end
    end
endmodule
```

```
always @(posedge clk) begin
    if (rst) begin
        registers[0] <= 16'b0;
        registers[1] <= 16'b0;
        registers[2] <= 16'b0;
        registers[3] <= 16'b0;
        registers[4] <= 16'b0;
        registers[5] <= 16'b0;
        registers[6] <= 16'b0;
        registers[7] <= 16'b0;
    end else if (write_en) begin
        registers[wr_addr] <= wr_data;
    end
end
endmodule
```

```

// Initialize and Stimulate Inputs
initial begin
    // Initialize Dump for GTKWave
    $dumpfile("register_file.vcd"); // VCD file name
    $dumpvars(0, tb_register_file); // Dump all variables in the testbench

    // Initialize Signals
    rst = 1;
    write_en = 0;
    wr_addr = 3'b000;
    rd_addr = 3'b000;
    wr_data = 16'h0000;

    // Monitor Signals: Automatically print whenever any listed signal changes
    $monitor("Time: %0t | rst: %b | write_en: %b | wr_addr: %b | wr_data: 0x%h | rd_addr: %b | rd_data: 0x%h",
        $time, rst, write_en, wr_addr, wr_data, rd_addr, rd_data);

    // Apply Reset
    #10;
    rst = 0;
    $display("Reset de-asserted at time %0t", $time);

    // Write Data to Register 3
    #10;
    write_en = 1;
    wr_addr = 3'b011;        // Register 3
    wr_data = 16'hABCD;      // Data to write
    $display("Writing 0x%h to Register %0d at time %0t", wr_data, wr_addr, $time);

```

```

// Write Data to Register 3
#10;
write_en = 1;
wr_addr = 3'b011;        // Register 3
wr_data = 16'hABCD;      // Data to write
$display("Writing 0x%h to Register %0d at time %0t", wr_data, wr_addr, $time);

// Wait for a Clock Edge to Perform Write
#10;
write_en = 0;
$display("Write enabled de-asserted at time %0t", $time);

// Read Back from Register 3
#10;
rd_addr = 3'b011;        // Register 3
$display("Reading from Register %0d at time %0t", rd_addr, $time);

// Wait for Read to Reflect
#10;

// Read from an Uninitialized Register (Register 0)
rd_addr = 3'b000;        // Register 0
$display("Reading from Register %0d at time %0t", rd_addr, $time);

// Wait for Read to Reflect
#10;

// Additional Operations (Optional)
// Example: Write to Register 5 and Read from it
write_en = 1;
wr_addr = 3'b101;        // Register 5
wr_data = 16'h;          // Data to write
// add 10 in register 5

```

```

// Read from an Uninitialized Register (Register 0)
rd_addr = 3'b000; // Register 0
$display("Reading from Register %0d at time %0t", rd_addr, $time);

// Wait for Read to Reflect
#10;

// Additional Operations (Optional)
// Example: Write to Register 5 and Read from it
write_en = 1;
wr_addr = 3'b101; // Register 5
// wr_data = 16'; // Data to write
// add 10 in register 5
wr_data = 16'h000A;
$display("Writing 0x%h to Register %0d at time %0t", wr_data, wr_addr, $time);

#10;
write_en = 0;
$display("Write enabled de-asserted at time %0t", $time);

rd_addr = 3'b101; // Register 5
$display("Reading from Register %0d at time %0t", rd_addr, $time);

#10;

// Finish Simulation
$display("Simulation completed at time %0t", $time);
$finish;

end

endmodule

```

OUTPUT:-

```

C:\iverilog\bin>vvp t
VCD info: dumpfile register_file.vcd opened for output.
Time: 0 | rst: 1 | write_en: 0 | wr_addr: 000 | wr_data: 0x0000 | rd_addr: 000 | rd_data: 0x0000
Reset de-asserted at time 10000
Time: 10000 | rst: 0 | write_en: 0 | wr_addr: 000 | wr_data: 0x0000 | rd_addr: 000 | rd_data: 0x0000
Writing 0xabcd to Register 3 at time 20000
Time: 20000 | rst: 0 | write_en: 1 | wr_addr: 011 | wr_data: 0xabcd | rd_addr: 000 | rd_data: 0x0000
Write enabled de-asserted at time 30000
Time: 30000 | rst: 0 | write_en: 0 | wr_addr: 011 | wr_data: 0xabcd | rd_addr: 000 | rd_data: 0x0000
Reading from Register 3 at time 40000
Time: 40000 | rst: 0 | write_en: 0 | wr_addr: 011 | wr_data: 0xabcd | rd_addr: 011 | rd_data: 0xabcd
Reading from Register 0 at time 50000
Time: 50000 | rst: 0 | write_en: 0 | wr_addr: 011 | wr_data: 0xabcd | rd_addr: 000 | rd_data: 0x0000
Writing 0x000a to Register 5 at time 60000
Time: 60000 | rst: 0 | write_en: 1 | wr_addr: 101 | wr_data: 0x000a | rd_addr: 000 | rd_data: 0x0000
Write enabled de-asserted at time 70000
Reading from Register 5 at time 70000
Time: 70000 | rst: 0 | write_en: 0 | wr_addr: 101 | wr_data: 0x000a | rd_addr: 101 | rd_data: 0x000a
Simulation completed at time 80000

```

